

Cách dùng Claude để học systems design thật sự hiệu quả

TLDR

Setup Claude như một tutor Socratic, học theo vòng lặp: **anh em đoán trước** → **Claude hỏi sâu hơn** → **neo vào sách** → **pattern recognition** → **lặp lại**. Kết hợp bottom-up (first principles từ sách) với top-down (production reality từ case study, tech blog, conference talk). Sau vài session anh em sẽ tự suy ra được cách một hệ thống mới hoạt động thay vì phải tra Google.

Ý tưởng cốt lõi

Học theo kiểu truyền thống là: đọc sách → hiểu lý thuyết → làm bài tập.
Cách này chán và hay quên.

Cách này khác: **bắt đầu bằng một hệ thống thực tế** (Redis, Kafka, Postgres...), rồi tự hỏi "tại sao nó được thiết kế vậy?" — Claude sẽ dẫn mình đi từng bước, và mỗi bước đều được neo vào sách giáo khoa cụ thể.

Kết quả: mình không chỉ biết Redis hoạt động như thế nào — mình hiểu *tại sao* nó được làm vậy, và có thể áp dụng logic đó để phân tích bất kỳ hệ thống nào khác.

Bottom-up và Top-down — cần cả hai

Có hai cách học systems design, và hầu hết mọi người chỉ làm một trong hai:

Bottom-up — học từ first principles: hash table, event loop, write-ahead log, TCP handshake. Hiểu *tại sao* mọi thứ hoạt động vậy ở mức cơ bản nhất. Nguồn: sách giáo khoa.

Top-down — học từ production reality: Netflix dùng eviction strategy gì, Cloudflare xử lý DDoS như thế nào, tại sao Discord chuyển từ Cassandra sang ScyllaDB. Hiểu *tại sao* đó va chạm với thực tế ở scale lớn. Nguồn: tech blog, case study, post-mortem, conference talk từ các công ty như Google, Meta, Stripe, Cloudflare...

Vấn đề:

- Bottom-up không có top-down → lý thuyết floating, không biết cái gì quan trọng ở scale thực tế
- Top-down không có bottom-up → biết pattern nhưng không thể reason về nó, không biết khi nào nên áp dụng

Workflow đúng: học first principles trước → dùng case study & tech blog để stress-test.

Khi đọc một case study, tech blog, hay xem conference talk, luôn tự hỏi: "*Cái này map về nguyên lý nào trong sách?*" Nếu không map được → chưa thật sự hiểu nguyên lý đó, cần quay lại sách trước.

Setup

Dùng Claude.ai Projects:

1. Tạo một Project mới
2. Upload sách vào project knowledge (PDF):
 - *The Algorithm Design Manual* — Skiena
 - *Designing Data-Intensive Applications* — Kleppmann
 - *Operating Systems: Three Easy Pieces* — Arpaci-Dusseau
 - *Computer Networking: Top-Down Approach* — Kurose/Ross
3. Copy nội dung file `CLAUDE.md` vào phần "Project Instructions"

Dùng Claude Code:

- Đẩy sách vào folder `books/`
- Copy `CLAUDE.md` vào root project

Cách bắt đầu một session

└ "I want to learn how to design [tên hệ thống]. Start from scratch."

Claude sẽ không dump thông tin. Nó sẽ hỏi một câu đơn giản để anh em tự suy nghĩ trước. Cứ trả lời thật — kể cả khi sai. Câu trả lời sai còn có giá trị hơn im lặng vì nó cho Claude biết chỗ nào cần khai thác.

Trong lúc học

- **Claude hỏi hai câu một lúc** → chỉ trả lời câu đầu, nói "one at a time." Câu thứ hai prime câu trả lời của mình và phá vỡ bài tập.
- **Claude giải thích trước khi hỏi** → nói "ask me to predict first."
- **Không biết gì về topic đó** → nói thẳng, Claude sẽ cho context tối thiểu rồi hỏi lại. Không bao giờ bị stuck.

Tại sao phải đoán trước?

Khi mình commit vào một dự đoán rồi thấy mình sai, não tạo ra *prediction error signal* — và đó là lúc long-term memory được ghi mạnh nhất. Đây là lý do làm bài tập trước khi đọc giải thích lại nhớ lâu hơn làm ngược lại. Claude trong workflow này là công cụ tạo ra prediction error liên tục, có hệ thống, được neo vào first principles thật sự.

Rewind

Edit lại tin nhắn cũ để branch context mới — dừng khi đi lạc vào tangent, hoặc Claude hỏi một câu tệ và anh em muốn retry. Claude không nhớ gì về nhánh bị xóa, fresh start hoàn toàn.

Kết thúc session

┆ *"Give me the reading assignment and ask me for my synthesis sentence."*

Claude sẽ cho biết đọc section nào cụ thể (ví dụ "OSTEP §33.1-33.4"). Sau đó tự viết một câu tóm tắt bằng lời của anh em — không copy của Claude. Đây là lúc mình buộc phải xử lý thông tin thay vì chỉ nhận nó thụ động.

Mở đầu session tiếp theo

┆ *"Quiz me on what we covered last time before we continue."*

Claude tự tìm lại session trước và hỏi 2-3 câu retrieval. Trả lời được → tiếp tục. Không trả lời được → đó là nội dung hôm nay, không cần đi tiếp.

Mỗi lần retrieve thành công là một lần memory trace được strengthen. Mỗi lần retrieve thất bại là tín hiệu chỗ nào cần học lại — tốt hơn là phát hiện ra điều đó trong phòng vẫn.

Tóm lại

Đừng	Nên
"Giải thích Redis cho mình"	"I want to design Redis from scratch"
Gật đầu khi Claude giải thích	Đoán trước khi Claude reveal
Học xong rồi thôi	Retrieval quiz đầu session sau
Xem case study để hiểu hệ thống	Dùng case study để test nguyên lý đã học
Để context bị lộn xộn	Rewind về điểm sạch khi cần
